11 / SQL RULES

SQL Reasoning Rules

Converting business questions into correct database logic

Purpose for the AI Assistant
The deterministic playbook the LangGraph tools follow to turn everyday questions into correct SQL. It fixes table/field choices, join paths, filters and aggregations so the LLM never improvises schema or violates a value rule.

1. How to use this document

Each rule below is a template, not free-form guidance. The router picks the department, this document supplies the exact SQL shape, and the tool fills in the resolved entity (item code, supplier, branch). Two rules are absolute and override any model instinct:

Two non-negotiable value rules
Inventory value = **SUM(stock.available_amount)** — never quantity × price.
Consumption value = **SUM(issuance.total_pric)** — never quantity × unit_price.

2. Question classification

Before mapping to SQL, every question is tagged with a class. The class decides how much reasoning and which template applies.

Class	Question shape	Example	Handling
Lookup	single fact about one entity	“What is stock of item X?”	Direct SELECT, entity resolved first
Aggregate	totals / counts / sums	“What is our inventory value?”	GROUP BY + SUM template
Ranking	top/worst N by a metric	“Which suppliers are most delayed?”	ORDER BY metric, LIMIT
Derived-metric	needs a formula / KPI	“What is supplier delay?”	Uses KPI library (Doc 14) formula
Diagnostic	why did X happen	“Why is item X critical?”	Multi-table join + comparison
Definition	what does a term mean	“What is hold stock?”	Business Dictionary, no SQL

Definition questions never hit the database
“What is supplier delay?” is a Definition question → answer from the Business Dictionary (“purchase_date – required_date, positive = late”). “What is our worst supplier delay?” is Derived-metric → run the SQL below.

3. STORES

“What is the current stock of item X?”

Business meaning	Physical quantity currently held for the item (optionally by branch).
Tables	stock
Fields	stock_qty, available_qty, hold_qty, branch
Join logic	resolve X → item_code (item_master)
Filters	item_code = :code
Aggregation	none (row-level) or SUM over branches

```
SQL concept
SELECT branch, stock_qty, available_qty, hold_qty
FROM stock WHERE item_code = :code;
```

“What is the available (usable) stock?”

Business meaning	Usable quantity = physical minus held.
Tables	stock
Fields	available_qty (= stock_qty - hold_qty)
Join logic	item_code join to item_master for name
Filters	item_code = :code
Aggregation	SUM(available_qty) if across branches

```
SQL concept
SELECT SUM(available_qty) FROM stock WHERE item_code = :code;
```

“How much is on hold / blocked?”

Business meaning	Reserved quantity that cannot be issued.
Tables	stock
Fields	hold_qty
Join logic	—
Filters	hold_qty > 0
Aggregation	SUM(hold_qty)

```
SQL concept
SELECT item_code, item, branch, hold_qty
FROM stock WHERE hold_qty > 0;
```

“What is our inventory value?”

Business meaning	Financial worth of usable stock in PKR.
Tables	stock
Fields	available_amount

Join logic	—
Filters	optional branch/category
Aggregation	SUM(available_amount)

```
SQL concept
SELECT SUM(available_amount) AS inventory_value_pkr
FROM stock; -- NEVER SUM(qty*price)
```

“Which items need reordering?”

Business meaning	Items whose usable stock is below their reorder level.
Tables	stock (or ab_items for richer logic)
Fields	available_qty, reorder_level
Join logic	—
Filters	reorder_level > 0 AND available_qty < reorder_level
Aggregation	count / list

```
SQL concept
SELECT item_code, item, branch, available_qty, reorder_level
FROM stock
WHERE reorder_level > 0 AND available_qty < reorder_level;
```

4. PURCHASE

“What is the supplier delay?”

Business meaning	Days a delivery ran past its required date.
Tables	purchase
Fields	purchase_date, required_date (delay_days)
Join logic	—
Filters	delay_days IS NOT NULL
Aggregation	AVG / MAX per supplier

```
SQL concept
SELECT supplier, ROUND(AVG(delay_days),1) AS avg_delay
FROM purchase GROUP BY supplier; -- delay = purchase_date - required_date
```

“Rank suppliers by reliability / delay.”

Business meaning	Order suppliers by average delay (or on-time rate).
Tables	purchase
Fields	supplier, delay_days, amount

Join logic	—
Filters	supplier IS NOT NULL
Aggregation	AVG(delay_days), COUNT(*), SUM(amount)

```
SQL concept
SELECT supplier, COUNT(*) po_lines,
ROUND(AVG(delay_days),1) avg_delay,
SUM(amount) spend
FROM purchase GROUP BY supplier
ORDER BY avg_delay DESC;
```

“What is our purchase value?”

Business meaning	Total value bought via POs in the window.
Tables	purchase
Fields	amount
Join logic	—
Filters	optional supplier/category/branch
Aggregation	SUM(amount)

```
SQL concept
SELECT SUM(amount) AS purchase_value_pkr FROM purchase;
```

5. IMPORTS

“How many items are on water?”

Business meaning	Shipments currently at sea (in transit).
Tables	import_shipment
Fields	current_status, total_value_pkr
Join logic	—
Filters	current_status = 'In Transit'
Aggregation	COUNT(*), SUM(total_value_pkr)

```
SQL concept
SELECT COUNT(*) shipments, SUM(total_value_pkr) value_pkr
FROM import_shipment WHERE current_status = 'In Transit';
```

“How many are sailing / awaiting sailing?”

Business meaning	IMPORT context: inbound shipments by departure state.
Tables	import_shipment
Fields	current_status

Join logic	—
Filters	current_status IN ('In Transit','Ready Awaiting Sailing')
Aggregation	COUNT grouped by status

```
SQL concept
SELECT current_status, COUNT(*)
FROM import_shipment
WHERE current_status IN ('In Transit','Ready Awaiting Sailing')
GROUP BY current_status; -- see Doc 12 for Imports-vs-Logistics routing
```

“What is the ETA delay / slippage?”

Business meaning	How far the latest ETA has slipped from the first ETA.
Tables	import_shipment
Fields	eta, eta_1 .. eta_4, eta_works
Join logic	—
Filters	eta_1 IS NOT NULL
Aggregation	AVG / per-shipment (eta - eta_1)

```
SQL concept
SELECT file_no, (eta - eta_1) AS slippage_days
FROM import_shipment WHERE eta_1 IS NOT NULL
ORDER BY slippage_days DESC;
```

6. LOGISTICS

“Which export shipments are delayed?”

Business meaning	Outbound shipments that arrived after their planned date.
Tables	logistics_shipment
Fields	actual_arrival, eta/planned, delay_status
Join logic	—
Filters	actual_arrival > planned_arrival
Aggregation	count / list, AVG(delay_days)

```
SQL concept
SELECT order_id, customer, (actual_arrival - etd_karachi) AS days
FROM logistics_shipment
WHERE actual_arrival IS NOT NULL
ORDER BY days DESC;
```

“What is freight performance / cost per kg?”

Business meaning	Shipping cost normalised by shipment weight.

Tables	logistics_shipment
Fields	total_shipping_cost, nw_kgs, gw_kgs
Join logic	—
Filters	nw_kgs > 0
Aggregation	SUM(cost)/SUM(weight), AVG(transit_time)

```
SQL concept
SELECT ROUND(SUM(total_shipping_cost)/NULLIF(SUM(nw_kgs),0),2) AS freight_per_kg,
ROUND(AVG(transit_time),1) AS avg_transit_days
FROM logistics_shipment;
```

7. ISSUANCE / CONSUMPTION

“What is our consumption value?”

Business meaning	Total value of material issued from stores.
Tables	issuance
Fields	total_pric
Join logic	—
Filters	optional date/branch/category
Aggregation	SUM(total_pric)

```
SQL concept
SELECT SUM(total_pric) AS consumption_pkr FROM issuance; -- NEVER qty*unit_price
```

“How much did a department consume?”

Business meaning	Consumption value grouped by department.
Tables	issuance
Fields	department, total_pric
Join logic	—
Filters	department = :dept (or all)
Aggregation	SUM(total_pric) GROUP BY department

```
SQL concept
SELECT department, SUM(total_pric) consumed_pkr, COUNT(*) lines
FROM issuance GROUP BY department ORDER BY consumed_pkr DESC;
```

“How much did a machine / job consume?”

Business meaning	Consumption attributed to a machine or production job.
Tables	issuance

Fields	machine, job_number, total_pric
Join logic	—
Filters	machine = :m OR job_number = :job
Aggregation	SUM(total_pric) GROUP BY machine/job

SQL concept

```
SELECT job_number, SUM(total_pric) cost_pkr
FROM issuance WHERE job_number IS NOT NULL
GROUP BY job_number ORDER BY cost_pkr DESC;
```

8. Guardrails the SQL layer enforces

- Resolve entities to keys (item_code, canonical supplier, branch) *before* building SQL — see Doc 15.
- Always parameterise (:code, :dept) — never string-concatenate user text into SQL.
- State the data window when quoting totals (issuance/purchase are windowed, not all-time).
- For value questions, use the pre-built views (Doc 16 / postgres_schema.sql) instead of ad-hoc SQL where one exists.
- If entity resolution is ambiguous, ask one clarifying question rather than guessing.